

DOCUMENTO ADR - EcoRoute

Registro de Decisiones Arquitectónicas

INTRODUCCIÓN

Este documento registra las decisiones tecnológicas y arquitectónicas tomadas durante el diseño del sistema EcoRoute. Cada registro sigue el formato estándar ADR: contexto del problema, decisión tomada, alternativas evaluadas, justificación y consecuencias. El objetivo es preservar el razonamiento detrás de cada elección para facilitar el mantenimiento, la replicación y la extensión futura del sistema.

Estado de los registros

Código	Título	Estado
ADR-001	Simulador de tráfico urbano	Aceptado
ADR-002	Algoritmo de aprendizaje por refuerzo	Aceptado
ADR-003	Framework de red neuronal	Aceptado
ADR-004	Interfaz de control del simulador	Aceptado
ADR-005	Función de recompensa híbrida escalar	Aceptado
ADR-006	Modelo de estimación de emisiones	Aceptado
ADR-007	Técnicas de estabilización del entrenamiento DQN	Aceptado
ADR-008	Interfaz estándar del entorno de RL	Aceptado
ADR-009	Estrategias de baseline para comparación	Aceptado
ADR-010	Alcance limitado a una intersección aislada	Aceptado
ADR-011	Formato de configuración centralizada	Aceptado
ADR-012	Herramientas de visualización y registro	Aceptado
ADR-013	Gestión del entorno de dependencias	Aceptado

ADR-001 — Simulador de Tráfico Urbano

Campo	Detalle
Fecha	Inicio del proyecto
Estado	Aceptado
Decider	Equipo de investigación EcoRoute

Contexto

El sistema requiere un entorno de simulación de tráfico urbano que permita modelar intersecciones reales, generar patrones de flujo vehicular variables y estimar emisiones de CO₂ a partir de datos de velocidad y aceleración. El simulador debe ser controlable externamente desde Python para permitir la interacción paso a paso con el agente de RL.

Decisión

Se utilizará SUMO (Simulation of Urban MObility) versión 1.18 o superior.

Alternativas Evaluadas

Alternativa	Descripción	Razón de descarte
VISSIM	Simulador comercial de tráfico ampliamente usado en ingeniería vial	De pago, licencia costosa, no accesible para investigación independiente; API de control limitada para RL
CityFlow	Simulador ligero diseñado específicamente para RL de control semafórico	No incluye modelos nativos de estimación de emisiones; menor fidelidad en modelado de intersecciones reales
CARLA	Simulador de conducción autónoma con entorno 3D	Orientado a percepción y conducción, no a gestión de tráfico; overhead computacional innecesario para el problema
AIMSUN	Simulador comercial de tráfico con módulo de emisiones	De pago; requiere licencia institucional; menor flexibilidad para integración con Python
Simulación propia	Desarrollar un simulador de tráfico desde cero	Inviabile en el tiempo del proyecto; no replicable ni validado científicamente

Justificación

SUMO es la única herramienta que cumple simultáneamente todos los criterios: es de código abierto, tiene soporte nativo de modelos de emisiones vehiculares, permite control externo en tiempo real mediante TraCI, cuenta con una comunidad activa y documentación extensa, y es ampliamente usado en investigación académica de RL aplicado a tráfico, lo que facilita la comparación con trabajos previos.

Consecuencias

Tipo	Descripción
Positiva	Acceso a modelos de emisiones integrados (HBEFA, PHEMlight) sin desarrollo adicional
Positiva	Comunidad activa y documentación extensa en español e inglés

Positiva	Compatibilidad validada con Python y TraCI para control en tiempo real
Negativa	Curva de aprendizaje de la API TraCI y el formato de archivos de red (.net.xml, .rou.xml)
Negativa	SUMO debe estar instalado como software independiente en el entorno; no es una biblioteca pip
Riesgo	Dependencia de versión específica de SUMO para compatibilidad con TraCI; cambios de API entre versiones pueden romper la integración

ADR-002 — Algoritmo de Aprendizaje por Refuerzo

Campo	Detalle
Fecha	Inicio del proyecto
Estado	Aceptado
Decider	Equipo de investigación EcoRoute

Contexto

El sistema necesita un agente de aprendizaje por refuerzo capaz de aprender políticas de control semafórico en un espacio de acciones discreto (mantener fase o cambiar a la siguiente) sin recibir reglas explícitas, aprendiendo únicamente por interacción con el entorno de simulación y una señal de recompensa.

Decisión

Se utilizará el algoritmo Deep Q-Network (DQN) con Experience Replay y Target Network.

Alternativas Evaluadas

Alternativa	Descripción	Razón de descarte
Q-Learning tabular	Método clásico de RL con tabla Q discreta	No escala a espacios de estado continuos o de alta dimensión como los vectores de densidad vehicular
PPO (Proximal Policy Optimization)	Algoritmo actor-crítico de política proximal	Mayor complejidad de implementación; mejor para espacios de acción continuos; el problema tiene acciones discretas simples
A3C (Asynchronous Advantage Actor-Critic)	Algoritmo actor-crítico distribuido	Requiere paralelismo y múltiples workers; añade complejidad innecesaria para una sola intersección

SAC (Soft Actor-Critic)	Algoritmo de máxima entropía para acciones continuas	Diseñado para espacios de acción continuos; no se adapta naturalmente al control semafórico discreto
Double DQN / Dueling DQN	Variantes mejoradas de DQN	Se consideran como extensiones futuras; DQN base es suficiente para demostrar la hipótesis central del proyecto

Justificación

DQN es el algoritmo de RL más establecido para espacios de acción discretos y de baja cardinalidad, exactamente el caso de EcoRoute (2 acciones: mantener o cambiar fase). Su combinación con Experience Replay y Target Network resuelve los problemas de correlación de datos y sobreestimación de valores Q que hacen inestable el entrenamiento en entornos de simulación. Además, existe amplia literatura académica de DQN aplicado a control semafórico, lo que facilita la comparación con el estado del arte.

Consecuencias

Tipo	Descripción
Positiva	Implementación relativamente directa con PyTorch; bien documentada en literatura académica
Positiva	Estabilidad de entrenamiento garantizada por Experience Replay y Target Network
Positiva	Resultados comparables con trabajos previos de control semafórico con RL
Negativa	DQN base puede ser superado por variantes como Double DQN o Dueling DQN en términos de rendimiento final
Negativa	Sensible a la elección de hiperparámetros; requiere ajuste experimental cuidadoso
Riesgo	Sobreestimación de valores Q puede persistir con DQN estándar; mitigable con Double DQN si los resultados son insatisfactorios

ADR-003 — Framework de Red Neuronal

Campo	Detalle
Fecha	Inicio del proyecto
Estado	Aceptado
Decider	Equipo de investigación EcoRoute

Contexto

El agente DQN requiere una red neuronal profunda para aproximar la función $Q(s,a)$. Se necesita un framework que permita definir la arquitectura de la red, realizar backpropagation automática, gestionar el optimizador y facilitar la depuración durante el desarrollo investigativo.

Decisión

Se utilizará PyTorch versión 2.0 o superior.

Alternativas Evaluadas

Alternativa	Descripción	Razón de descarte
TensorFlow / Keras	Framework de Google para deep learning	API de Keras más abstracta dificulta el acceso directo a tensores durante el debugging; mayor verbosidad para investigación experimental
JAX	Framework de Google basado en XLA con diferenciación automática	Curva de aprendizaje muy alta; ecosistema más pequeño para RL; menos bibliotecas compatibles
Stable-Baselines3	Biblioteca de RL de alto nivel sobre PyTorch	Abstrae demasiado la implementación interna del DQN; impide modificar la arquitectura de recompensa y el replay buffer para los objetivos investigativos del proyecto
RLlib	Framework distribuido de RL de Ray	Orientado a producción y escala; overhead innecesario para un experimento de investigación en una intersección

Justificación

PyTorch es el estándar de facto en investigación de deep learning y RL. Su modo de ejecución eager (define-by-run) facilita el debugging interactivo, lo cual es crítico en investigación donde la arquitectura de la red y la función de pérdida se iteran frecuentemente. Además, la mayoría de implementaciones académicas de DQN para control de tráfico están en PyTorch, lo que facilita la referencia cruzada con trabajos previos.

Consecuencias

Tipo	Descripción
Positiva	Debugging sencillo con ejecución eager; inspección directa de tensores en cualquier punto
Positiva	Amplio ecosistema: torchvision, torch.optim, soporte CUDA nativo
Positiva	Compatible con conda y pip; instalación reproducible en Linux y Windows

Negativa	Requiere implementación manual del loop de entrenamiento DQN (no hay abstracción de alto nivel)
Negativa	Mayor código boilerplate comparado con Keras para tareas simples

ADR-004 — Interfaz de Control del Simulador

Campo	Detalle
Fecha	Inicio del proyecto
Estado	Aceptado
Decider	Equipo de investigación EcoRoute

Contexto

El agente DQN necesita interactuar con SUMO paso a paso: leer el estado del tráfico en cada instante, enviar acciones semafóricas y recibir las consecuencias de esas acciones. Se requiere una interfaz que permita este control en tiempo real desde Python.

Decisión

Se utilizará TraCI (Traffic Control Interface), la API oficial de Python incluida con SUMO.

Alternativas Evaluadas

Alternativa	Descripción	Razón de descarte
libsumo	Binding C++ de SUMO con interfaz Python	Mayor rendimiento que TraCI pero menor estabilidad en Python; documentación más escasa; mayor riesgo de bugs en la integración
SUMO-RL	Biblioteca de alto nivel sobre TraCI para RL	Abstrae la interacción con SUMO pero limita el acceso a datos de emisiones a nivel de vehículo individual necesarios para el modelo VSP/COPERT
Archivos de traza offline	Generar datos de simulación y leerlos como batch	No permite interacción en tiempo real; el agente no puede influir en el flujo de la simulación

Justificación

TraCI es la interfaz oficial y más madura para controlar SUMO desde Python. Permite acceso en tiempo real a todos los datos de la simulación (velocidad, aceleración, densidad, fase semafórica) y control directo de las fases de los semáforos, exactamente lo que el agente DQN necesita. Su uso está documentado en decenas de trabajos académicos de RL para tráfico.

Consecuencias

Tipo	Descripción
Positiva	Acceso completo a todos los datos de SUMO en cada paso de simulación
Positiva	Control directo de fases semafóricas con un solo comando Python
Positiva	Documentación oficial extensa y ejemplos de uso en Python
Negativa	Comunicación vía socket TCP introduce latencia mínima por paso de simulación
Negativa	TraCI requiere que SUMO esté corriendo como proceso separado antes de conectar
Riesgo	Cambios en la API de TraCI entre versiones de SUMO pueden requerir actualizaciones del código

ADR-005 — Función de Recompensa Híbrida Escalar

Campo	Detalle
Fecha	Diseño del agente
Estado	Aceptado
Decider	Equipo de investigación EcoRoute

Contexto

EcoRoute tiene dos objetivos simultáneos y parcialmente en tensión: reducir las emisiones de CO₂ y mantener la fluidez del tráfico. Se debe definir cómo el agente recibe retroalimentación que balancee ambos objetivos durante el entrenamiento.

Decisión

Se utilizará una función de recompensa híbrida escalar:

$$R = -\alpha \cdot (\text{espera acumulada}) - \beta \cdot (\text{emisiones CO}_2) + \gamma \cdot (\text{throughput vehicular})$$

con pesos α , β , γ configurables desde config.yaml.

Alternativas Evaluadas

Alternativa	Descripción	Razón de descarte
Recompensa solo por fluidez	Penalizar únicamente el tiempo de espera	Ignora el objetivo de reducción de emisiones; no responde a la pregunta de investigación del proyecto
Recompensa solo por emisiones	Penalizar únicamente el CO ₂ estimado	Puede degradar la movilidad hasta niveles inaceptables; no es una solución práctica para gestión urbana

Optimización multi-objetivo (Pareto)	Mantener múltiples objetivos separados y optimizar el frente de Pareto	Requiere algoritmos de RL multi-objetivo más complejos (MORL); dificulta la comparación con baselines estándar; fuera del alcance del proyecto
Recompensa lexicográfica	Priorizar un objetivo sobre otro de forma jerárquica	Introduce rigidez en el balance; no permite ajustar fácilmente la importancia relativa de cada objetivo

Justificación

La función escalar ponderada es el enfoque más pragmático para un agente DQN estándar: permite entrenar un único agente con un solo valor de recompensa por paso, mantiene la compatibilidad con el algoritmo DQN sin modificaciones, y hace que el balance entre objetivos sea un hiperparámetro experimental ajustable (α , β , γ), lo que permite explorar distintos compromisos entre fluidez y sostenibilidad.

Consecuencias

Tipo	Descripción
Positiva	Compatible con DQN estándar sin modificaciones al algoritmo
Positiva	Balance entre objetivos ajustable como hiperparámetro experimental
Positiva	Permite estudiar el efecto de distintos pesos sobre el comportamiento del agente
Negativa	La elección de α , β , γ afecta significativamente el resultado; requiere búsqueda de hiperparámetros
Negativa	No garantiza optimalidad en el frente de Pareto; el agente puede sacrificar un objetivo por el otro según los pesos
Riesgo	Pesos mal calibrados pueden hacer que el agente ignore completamente uno de los objetivos

ADR-006 — Modelo de Estimación de Emisiones

Campo	Detalle
Fecha	Diseño del módulo de emisiones
Estado	Aceptado
Decider	Equipo de investigación EcoRoute

Contexto

La función de recompensa penaliza las emisiones de CO₂. Se necesita un modelo que estime estas emisiones por paso de simulación a partir de los datos disponibles en SUMO (velocidad instantánea y aceleración vehicular), sin requerir sensores físicos externos.

Decisión

Se implementarán dos modelos intercambiables: VSP (Vehicle Specific Power) como modelo principal y COPERT simplificado como alternativa, seleccionables desde config.yaml.

Alternativas Evaluadas

Alternativa	Descripción	Razón de descarte
HBEFA (Handbook Emission Factors)	Base de datos europea de factores de emisión por categoría vehicular	Requiere clasificación detallada de la flota vehicular; datos no siempre disponibles para ciudades latinoamericanas
PHEMlight	Modelo de emisiones integrado en SUMO basado en datos de pruebas de vehículos europeos	Mayor precisión pero requiere archivos de parámetros por tipo de vehículo difíciles de calibrar; overhead innecesario para los fines comparativos del proyecto
Emisiones directas de SUMO	Usar los valores de emisiones que SUMO calcula internamente	Dependencia total de la configuración interna de SUMO; menor control sobre el modelo usado; dificulta la explicación metodológica
Modelo de velocidad media únicamente	Estimar emisiones solo a partir de la velocidad promedio por carril	No captura el efecto de la aceleración y el frenado, que son la principal fuente de emisiones en intersecciones urbanas

Justificación

VSP es un modelo ampliamente validado en literatura académica de emisiones de tráfico urbano que captura el efecto de la aceleración y la desaceleración, los principales factores de emisión en intersecciones. Su fórmula es computacionalmente simple ($VSP = v \cdot (a + g \cdot \sin(\theta) + 0.132) + 0.000302 \cdot v^3$) y usa datos disponibles directamente en SUMO. COPERT se incluye como alternativa para validar la robustez de los resultados ante distintos modelos de emisiones.

Consecuencias

Tipo	Descripción
Positiva	VSP captura el efecto de aceleración y frenado, crítico en intersecciones semaforizadas

Positiva	Modelo intercambiable permite validar resultados con dos metodologías distintas
Positiva	Fórmula simple y transparente, reproducible sin herramientas externas
Negativa	VSP simplificado asume pendiente cero ($\theta = 0$); puede subestimar emisiones en ciudades con topografía irregular como Bogotá
Negativa	No diferencia entre tipos de vehículo (gasolina, diésel, eléctrico); asume flota homogénea
Riesgo	Los valores absolutos de CO ₂ estimado pueden diferir de mediciones reales; los resultados son válidos para comparación relativa entre políticas, no como medición absoluta

ADR-007 — Técnicas de Estabilización del Entrenamiento DQN

Campo	Detalle
Fecha	Diseño del agente
Estado	Aceptado
Decider	Equipo de investigación EcoRoute

Contexto

El entrenamiento de una red neuronal con datos generados por la misma política que se está aprendiendo introduce dos problemas conocidos: correlación entre muestras consecutivas (que viola el supuesto de independencia del SGD) y sobreestimación de los valores Q objetivo (que genera divergencia del entrenamiento). Se deben implementar técnicas que mitiguen estos problemas.

Decisión

Se implementarán Experience Replay con buffer circular y Target Network con actualización periódica cada N pasos.

Alternativas Evaluadas

Alternativa	Descripción	Razón de descarte
Entrenamiento online sin replay	Actualizar la red con cada transición inmediatamente	Alta correlación entre muestras consecutivas; inestabilidad del entrenamiento documentada en literatura
Prioritized Experience Replay (PER)	Asignar mayor probabilidad de muestreo a transiciones con mayor error TD	Mayor complejidad de implementación (árbol de suma); beneficio marginal para

		el alcance del proyecto; considerado como extensión futura
Actualización continua del target	Usar soft updates ($\tau \cdot \theta + (1-\tau) \cdot \theta'$) en lugar de copia periódica	Alternativa válida; se descarta en favor de la copia periódica por simplicidad y mayor facilidad de interpretación del hiperparámetro N
Sin target network	Usar la misma red para calcular Q objetivo y Q actual	Genera el problema de "chasing a moving target"; divergencia documentada; descartado por inestabilidad

Justificación

Experience Replay y Target Network son las dos técnicas fundamentales introducidas en el DQN original de DeepMind (Mnih et al., 2015) y son consideradas requisitos mínimos para un entrenamiento DQN estable. Su implementación es directa, bien documentada y suficiente para el alcance del proyecto. PER y soft updates se reservan como mejoras opcionales si el entrenamiento base muestra inestabilidad.

Consecuencias

Tipo	Descripción
Positiva	Rompe la correlación entre muestras consecutivas mediante muestreo aleatorio del buffer
Positiva	Estabiliza los valores Q objetivo al fijar los pesos de la target network durante N pasos
Positiva	Implementación directa con bajo overhead computacional
Negativa	El buffer de replay consume memoria RAM proporcional a su capacidad (configurable)
Negativa	El entrenamiento no comienza hasta que el buffer alcanza el tamaño mínimo configurado
Riesgo	El valor de N (frecuencia de actualización del target) afecta significativamente la estabilidad; requiere ajuste experimental

ADR-008 — Interfaz Estándar del Entorno de RL

Campo	Detalle
Fecha	Diseño del módulo de entorno

Estado	Aceptado
Decider	Equipo de investigación EcoRoute

Contexto

El módulo de entorno debe exponer una interfaz que permita al agente DQN interactuar con él de forma estandarizada, independientemente de los detalles internos de SUMO y TraCI.

Decisión

El módulo de entorno implementará la interfaz estándar de Gymnasium (sucesor de OpenAI Gym): métodos `reset()`, `step(action)`, `render()` y `close()`.

Alternativas Evaluadas

Alternativa	Descripción	Razón de descarte
Interfaz propia	Definir una API de entorno personalizada para EcoRoute	Impide el uso de bibliotecas de RL estándar (Stable-Baselines3, RLlib) en el futuro; menor interoperabilidad
Interfaz de SUMO-RL	Usar la interfaz que define la biblioteca SUMO-RL	Dependencia adicional; menor control sobre los datos de estado y recompensa que el proyecto necesita

Justificación

La interfaz Gymnasium es el estándar de facto en investigación de RL. Implementarla garantiza que el entorno de EcoRoute sea compatible con cualquier biblioteca de RL en el futuro, facilita la extensión a multi-agente con PettingZoo (extensión multi-agente de Gymnasium) y hace el código más legible para cualquier investigador familiarizado con el ecosistema de RL en Python.

Consecuencias

Tipo	Descripción
Positiva	Compatibilidad futura con Stable-Baselines3, RLlib y otras bibliotecas de RL
Positiva	Extensión a multi-agente facilitada mediante PettingZoo
Positiva	Código más legible y familiar para la comunidad de investigación en RL
Negativa	Requiere adaptar los datos de SUMO al formato de espacios de observación y acción de Gymnasium (<code>spaces.Box</code> , <code>spaces.Discrete</code>)

ADR-009 — Estrategias de Baseline para Comparación

Campo	Detalle
-------	---------

Fecha	Diseño del módulo de entrenamiento
Estado	Aceptado
Decider	Equipo de investigación EcoRoute

Contexto

Para demostrar que el agente DQN mejora el control semafórico, se necesitan estrategias de referencia con las que comparar sus resultados. Estas estrategias deben representar el estado actual del arte en control semafórico real y ejecutarse sobre los mismos escenarios que el agente.

Decisión

Se implementarán dos baselines: control de tiempo fijo (ciclos predefinidos) y control adaptativo clásico actuado por presencia vehicular.

Alternativas Evaluadas

Alternativa	Descripción	Razón de descarte
SCOOT	Sistema adaptativo comercial de control semafórico en tiempo real	Propietario; requiere infraestructura física especializada; no integrable en SUMO para comparación directa
SCATS	Sistema australiano de control semafórico adaptativo	Mismo problema que SCOOT: propietario y dependiente de infraestructura física
Política aleatoria	Seleccionar acciones semafóricas al azar	Poco representativa del estado real del arte; no aporta valor comparativo significativo
Solo tiempo fijo	Un único baseline de referencia	Insuficiente para demostrar mejora sobre distintos niveles de sofisticación del control semafórico

Justificación

El control de tiempo fijo representa el estado actual de la mayoría de intersecciones urbanas en ciudades latinoamericanas y es la referencia mínima que EcoRoute debe superar. El control adaptativo clásico (actuado por presencia) representa el siguiente nivel de sofisticación, sin aprendizaje, que demuestra que la mejora del DQN no se debe simplemente a la adaptación al flujo, sino al aprendizaje de políticas óptimas considerando emisiones.

Consecuencias

Tipo Descripción

Positiva Dos niveles de referencia permiten demostrar la mejora incremental del DQN

Tipo	Descripción
Positiva	Ambos baselines son implementables directamente en SUMO sin herramientas externas
Positiva	Comparación directa en el mismo escenario garantiza validez experimental
Negativa	Los baselines no incluyen sistemas comerciales avanzados (SCOOT, SCATS); la comparación con el estado del arte real queda limitada a literatura

ADR-010 — Alcance Limitado a una Intersección Aislada

Campo	Detalle
Fecha	Definición del alcance del proyecto
Estado	Aceptado
Decider	Equipo de investigación EcoRoute

Contexto

El control de tráfico puede abordarse desde una intersección aislada hasta redes completas de intersecciones coordinadas. Definir el alcance correcto es crítico para la viabilidad del proyecto dentro de sus restricciones de tiempo y recursos.

Decisión

El alcance inicial de EcoRoute cubre el control de una única intersección urbana aislada. La extensión a redes multi-intersección se define como trabajo futuro.

Alternativas Evaluadas

Alternativa	Descripción	Razón de descarte
Red de 2-4 intersecciones coordinadas	Controlar múltiples intersecciones con agentes independientes	Multiplica la complejidad de la simulación SUMO, del espacio de estados y del entrenamiento; dificulta el diagnóstico de fallos
Arquitectura multi-agente desde el inicio	Diseñar directamente para múltiples agentes coordinados	Requiere algoritmos MARL (Multi-Agent RL) significativamente más complejos; inviable en el tiempo disponible
Corredor semafórico (ola verde)	Coordinar una secuencia lineal de intersecciones	Caso especial de multi-agente; mismo problema de complejidad

Justificación

Limitar el alcance a una intersección aislada permite validar la hipótesis central del proyecto (DQN reduce huella de carbono sin deteriorar movilidad) de forma controlada, con menor complejidad de simulación y depuración más directa. La arquitectura modular garantiza que la extensión futura a multi-agente sea posible sin rediseño. Es el enfoque estándar en la literatura de RL para tráfico: demostrar viabilidad en caso simple antes de escalar.

Consecuencias

Tipo	Descripción
Positiva	Menor complejidad de implementación, depuración y entrenamiento
Positiva	Resultados más interpretables y directamente atribuibles al comportamiento del agente
Positiva	Arquitectura modular preparada para extensión futura sin rediseño
Negativa	Los resultados no capturan efectos de coordinación entre intersecciones (ondas verdes, redistribución de flujo)
Negativa	La generalización a redes reales requiere trabajo adicional significativo

ADR-011 — Formato de Configuración Centralizada

Campo	Detalle
Fecha	Diseño de la estructura del proyecto
Estado	Aceptado
Decider	Equipo de investigación EcoRoute

Contexto

El sistema tiene múltiples parámetros configurables: hiperparámetros del agente, pesos de la función de recompensa, parámetros de simulación, semillas aleatorias y rutas de archivos. Se necesita una estrategia para gestionar estos parámetros de forma centralizada y reproducible.

Decisión

Todos los parámetros configurables se centralizarán en un único archivo config.yaml, cargado al inicio por el módulo ConfigLoader. Ningún módulo tendrá parámetros hardcodeados.

Alternativas Evaluadas

Alternativa	Descripción	Razón de descarte
-------------	-------------	-------------------

Argumentos de línea de comandos (argparse)	Pasar parámetros directamente por CLI	Dificulta la reproducibilidad: los comandos exactos deben documentarse manualmente; no hay registro persistente de la configuración usada
Variables de entorno	Configurar parámetros mediante variables de entorno del SO	No versionable con Git de forma natural; difícil de auditar y reproducir entre entornos
Archivo .json	Usar JSON en lugar de YAML	YAML es más legible para humanos y soporta comentarios, críticos para documentar el significado de cada parámetro
Parámetros hardcodeados en el código	Definir parámetros directamente en el código fuente	Imposible reproducir experimentos con distintas configuraciones sin modificar el código; contrario a buenas prácticas científicas

Justificación

Un único archivo YAML centralizado garantiza que cada experimento quede completamente definido por su archivo de configuración, que puede versionarse con Git y archivarse junto con los resultados. Esto es esencial para la reproducibilidad científica del proyecto y facilita la ejecución de barridos de hiperparámetros simplemente modificando el YAML.

Consecuencias

Tipo	Descripción
Positiva	Reproducibilidad total: cada experimento queda definido por su config.yaml
Positiva	Facilita barridos de hiperparámetros sin modificar el código fuente
Positiva	El archivo de configuración puede versionarse con Git junto al código
Negativa	Requiere que todos los módulos dependan del ConfigLoader; introduce una dependencia transversal

ADR-012 — Herramientas de Visualización y Registro

Campo	Detalle
Fecha	Diseño del módulo de evaluación
Estado	Aceptado
Decider	Equipo de investigación EcoRoute

Contexto

El sistema debe registrar métricas por episodio y generar visualizaciones que permitan al investigador interpretar el comportamiento del agente y comparar políticas. Se necesitan herramientas que sean estándar, reproducibles y exportables.

Decisión

Se utilizará pandas para el registro y exportación de métricas a CSV, matplotlib para gráficas de convergencia y comparativas, y seaborn para mapas de calor de densidad vehicular.

Alternativas Evaluadas

Alternativa	Descripción	Razón de descarte
TensorBoard	Herramienta de visualización de entrenamiento de TensorFlow	Requiere servidor local activo; menos conveniente para exportar gráficas estáticas para reportes científicos
Weights & Biases (WandB)	Plataforma cloud de seguimiento de experimentos	Requiere cuenta y conexión a internet; dependencia externa innecesaria para el alcance del proyecto
MLflow	Plataforma de gestión del ciclo de vida de modelos ML	Overhead de infraestructura excesivo para un proyecto de investigación de una intersección
Plotly	Biblioteca de gráficas interactivas en Python	Gráficas interactivas no son necesarias para reportes estáticos; mayor complejidad sin beneficio adicional

Justificación

pandas + matplotlib + seaborn es el stack estándar de análisis de datos en Python académico. No requieren servicios externos, generan archivos exportables en PNG/PDF directamente, y son dominados por la mayoría de investigadores en el área. Su uso garantiza que cualquier investigador pueda reproducir las visualizaciones sin instalar herramientas adicionales más allá del entorno conda/pip del proyecto.

Consecuencias

Tipo	Descripción
Positiva	Sin dependencias de servicios externos ni conexión a internet
Positiva	Exportación directa a PNG y PDF para inclusión en reportes y artículos
Positiva	Stack familiar para la comunidad de investigación en Python

Negativa	Sin visualización en tiempo real durante el entrenamiento (solo post-episodio)
Negativa	Sin dashboard interactivo; exploración de resultados limitada a gráficas estáticas

ADR-013 — Gestión del Entorno de Dependencias

Campo	Detalle
Fecha	Configuración del proyecto
Estado	Aceptado
Decider	Equipo de investigación EcoRoute

Contexto

El sistema tiene múltiples dependencias con versiones específicas (Python, PyTorch, SUMO, TraCI, pandas, matplotlib). Se necesita una estrategia para gestionar estas dependencias de forma que el entorno sea reproducible en distintas máquinas y sistemas operativos.

Decisión

Se documentarán las dependencias en dos formatos paralelos: `environment.yml` para conda (recomendado) y `requirements.txt` para pip (alternativo), con versiones fijadas para todas las dependencias críticas.

Alternativas Evaluadas

Alternativa	Descripción	Razón de descarte
Solo requirements.txt (pip)	Gestionar dependencias únicamente con pip	pip no gestiona dependencias del sistema (como CUDA) ni resuelve conflictos entre paquetes con C extensions tan bien como conda
Docker	Contenedor con todas las dependencias preinstaladas	Mayor portabilidad pero overhead de aprendizaje; acceso a GPU desde Docker requiere configuración adicional (nvidia-docker); excesivo para el alcance del proyecto
Poetry	Gestor de dependencias moderno para Python	Excelente para proyectos de producción; menos familiar en el contexto de investigación científica en Python
Sin fijación de versiones	Instalar las últimas versiones disponibles	Riesgo de incompatibilidades entre versiones de PyTorch, CUDA y SUMO que rompan la reproducibilidad

Justificación

Conda es el gestor de entornos estándar en la comunidad de investigación científica en Python. Gestiona tanto paquetes Python como dependencias del sistema (incluyendo CUDA para GPU) y resuelve conflictos de forma más robusta que pip solo. El archivo environment.yml puede versionarse con Git y reproducir el entorno exacto en cualquier máquina con un único comando. requirements.txt se incluye como alternativa para entornos donde conda no esté disponible.

Consecuencias

Tipo	Descripción
Positiva	Reproducción del entorno completo con un único comando: conda env create -f environment.yml
Positiva	Gestión de dependencias del sistema (CUDA, compiladores) incluida en conda
Positiva	Archivo versionable con Git; garantiza reproducibilidad a largo plazo
Negativa	Conda puede ser lento resolviendo dependencias en algunos entornos
Negativa	Requiere instalación previa de conda/miniconda en la máquina del investigador
Riesgo	SUMO debe instalarse por separado del entorno conda (instalador oficial o gestor de paquetes del SO); no está disponible como paquete conda en todos los canales

RESUMEN DE DECISIONES

ADR	Decisión tomada	Alternativa principal descartada	Razón clave del descarte
ADR-001	SUMO como simulador	VISSIM	Propietario y costoso
ADR-002	Algoritmo DQN	PPO	Espacio de acciones discreto simple
ADR-003	PyTorch	TensorFlow/Keras	Mayor flexibilidad para investigación
ADR-004	TraCI como interfaz	libsumo	Mayor estabilidad y documentación
ADR-005	Recompensa híbrida escalar	Optimización multi-objetivo	Compatible con DQN estándar
ADR-006	VSP + COPERT intercambiables	PHEMlight	Simplicidad y datos disponibles en SUMO

ADR-007	Experience Replay + Target Network	Entrenamiento online	Estabilidad del entrenamiento
ADR-008	Interfaz Gymnasium	Interfaz propia	Interoperabilidad con ecosistema RL
ADR-009	Tiempo fijo + adaptativo clásico	SCOOT / SCATS	Propietarios y no integrables en SUMO
ADR-010	Una intersección aislada	Red multi-intersección	Viabilidad y validación de hipótesis central
ADR-011	config.yaml centralizado	argparse / hardcoded	Reproducibilidad científica
ADR-012	pandas + matplotlib + seaborn	WandB / TensorBoard	Sin dependencias externas
ADR-013	conda + requirements.txt	Solo pip / Docker	Estándar en investigación científica Python